

00660121 - Medical Diagnostics HW1: Multiple Linear Regression & Gradient Descent

TA in charge: Mattan Hoory*
Faculty of Biotechnology and Food Engineering
Technion - Israel Institute of Technology

Spring 2026

Submission Due: 12.07.26

Table of contents

1	Aim of the assignment	2
2	Dataset & background	3
2.1	Mini-batch gradient descent	4
2.2	Writing and running code	5
2.2.1	Google Colab / Kaggle	6
3	Tasks	7
3.1	Task 1 – Implement the model	7
3.2	Task 2 – Train the model	8
3.3	Task 3 – Evaluation & analysis	8

*Thanks to Prof. Roei Amit and the Technion Faculty of Biotechnology & Food Engineering for their support.

3.4	Task 4 – From Regression to a “Toy” Classifier	9
3.5	Task 5 – Theoretical questions	13
4	Submission Guidelines	14
4.1	Written report guidelines	14
5	Grading rubric	16
6	Academic integrity & help	16
7	FAQ	17

1 Aim of the assignment

In lecture 1 we defined supervised learning and saw the linear-regression model.

This homework lets you:

- Implement *multiple* (multi-input:one-output) linear regression **from scratch** with gradient descent
- Apply it to a **mini-GEO gene-expression subset** (landmark → a **single** target gene)
- Measure and reflect on the model’s limits.

Some of the code you write here might be reused when we extend to neural networks in HW2.

Work in pairs. One submission per pair. **Due date:** 12.07.2026, 23:59. **Late submission:** 10% penalty per day. **Submission:** Through moodle, ZIP file with code and report, see details below.

- *Steps* are marked *A.*, *B.*, *C.* etc. These require you to write code, and run it. to obtain results for the report you write.
- *Questions/Plots* required for the report are marked numerically *3.1.1*, *3.1.2*, etc. These require you to answer the questions or use code to create the plots, and then include them in your report. You can answer the questions as a paragraph, or as a list of answers, but make sure to include all the required information.
- Theoretical questions appear in section 5, the answers to these questions should be included in your report as well.

2 Dataset & background

File / folder	Purpose
gene_expression_regression.csv	mini-GEO gene-expression dataset containing the landmark genes and one target gene from the original D-GEX dataset. The target gene is in the last column.

Click the link above to download the dataset, you will need to unzip it to access the `gene_expression_regression.csv` file. Pay attention to the file format, it is a CSV file with a header row and column. The last column is the target gene expression value, and the other columns are the landmark genes' expression values. The data is already preprocessed and ready for use.

Reference:

2.1 Mini-batch gradient descent

In the tutorial we saw plain **(full-)batch gradient descent**: at every step we use **all** n training samples to compute the gradient of the loss and take one step downhill,

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta),$$

where η is the **learning rate**. This is accurate but can be slow and memory-heavy when n is large, since *every* update has to pass over the entire dataset.

Mini-batch gradient descent is the middle ground we use in this assignment. Instead of the whole dataset (or a single sample) per step, we split the training data into small chunks called **batches** of size B , and update the parameters once per batch:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L_{\mathcal{B}}(\theta), \quad L_{\mathcal{B}}(\theta) = \frac{1}{B} \sum_{i \in \mathcal{B}} \ell(h_{\theta}(x_i), y_i),$$

i.e. the loss (and therefore its gradient) is averaged over the B samples in the current batch $\mathcal{B}$ rather than over all n samples. One full sweep over all batches — so that every sample has been used exactly once — is called an **epoch**, and we train for several epochs.

The **batch size** B is a hyperparameter you choose:

- $B = n \rightarrow$ **(full-)batch gradient descent**: one update per epoch, smooth but slow.
- $B = 1 \rightarrow$ **stochastic gradient descent (SGD)**: one update per sample, very noisy.
- $1 < B < n \rightarrow$ **mini-batch**: the usual practical choice (e.g. 16, 32, 64).

Why we use it:

- More frequent updates and better memory use on large datasets.
- The noise from using only a subset of the data can help the optimiser escape poor local minima / saddle points.

What to watch out for:

- The noisy gradient makes the loss curve oscillate rather than decrease smoothly.
- Results are sensitive to the learning rate η and batch size B ; both usually need tuning.

A typical training loop looks like:

```
initialize parameters
for each epoch:
    shuffle the training data
    for each batch B in training_data:
        compute predictions   $h_{\theta}(x)$  for  $x$  in B
        compute batch loss    $L_B(\theta)$            # e.g. MSE over the batch
        compute gradient      $\nabla_{\theta} L_B(\theta)$ 
        update parameters     $\theta \leftarrow \theta - \eta \nabla_{\theta} L_B(\theta)$ 
    (optional) evaluate on the validation set
```

This is exactly the training loop you will implement in **Task 1** and use in **Task 2**, where you get to pick the learning rate and batch size yourself.

2.2 Writing and running code

To write and execute the code, you can use either **VScode** with an **Anaconda** environment, **Google Colab** / **Kaggle** , or any other environment you see fit. see HW0 for details.

2.2.1 Google Colab / Kaggle

1. Create a new notebook.
2. Upload the `gene_expression_regression.csv` file to the notebook. For *Kaggle* you can use the dataset I've uploaded [here](#)

3 Tasks

A. Read the (1) Introduction, and (2) Dataset (up until and including 2.2) chapters from the publication “Gene expression inference with deep learning” by Chen et al. (2016) to understand the dataset and the task at hand.

B. (**Sanity Check**). Using code, open the file `gene_expression_regression.csv`, print the first 5 rows, to check that the data is loaded correctly.

3.0.1 What do the columns represent? What do the rows represent?

3.0.2 How many samples and features are there?

3.1 Task 1 – Implement the model

A. Implement a function to fit a **linear regression model** using **mini-batch gradient descent** and **mean squared error (MSE)** loss. Use the following notation:

$$X \in \mathbb{R}^{n \times m}$$

$$Y \in \mathbb{R}^n$$

(Last column is the target gene)

Implement this part yourself (a full ‘training-loop’ like what we have seen in the tutorials), do **not** use imported optimisers. In the tutorials we have seen an example for a single-variable linear regression, here you will implement a **multiple linear regression** model, which is a generalization of the single-variable case.

3.1.1 What are the inputs and outputs of the model (h_θ)?

3.1.1.1 What parameters are learned during training?

3.1.1.2 What parameters are fixed during training?

3.1.1.3 What is the weight vector (denoted w), what is its shape/size?

3.1.2 What is the loss function used for training? Write down the equation for the loss function.

3.1.3 What is the gradient of the loss function with respect to the model parameters? Write down the equation for the gradient.

3.1.4 What is the update rule for the model parameters? Write down the equation for the update rule.

3.2 Task 2 – Train the model

A. Split data 80 % / 10 % / 10 % into **train** / **val** / **test**, for this part only you may use `sklearn.model_selection.train_test_split` (with `random_state=42`). You can read about using it [here](#).

Note: The split is fixed for reproducibility

B. Decide on a learning rate and batch size.

C. Plot the losses vs epochs for both train and validation sets.

3.3 Task 3 – Evaluation & analysis

A. Experiment and compare different learning rates and batch sizes.

B. Evaluate the model on the test set using **mean squared error (MSE)** and **Pearson correlation** between your predictions ($h(X) = \hat{Y}$) and the true values (Y).

C. Plot the **predicted vs true values** for the test set, and add the line $y = x$ to the plot.

3.3.1 Report your results and findings. Which learning rate and batch size worked best?

How did you determine that? (Add plots if necessary.)

3.3.2 Report the best **final test MSE** and **Pearson correlation** (one number each).

Mention any changes you made to the model or training process that improved the results.

3.3.3 Looking at the predicted vs true plot you made in C ., explain what the line $y = x$ represents and what it tells you about the quality of your predictions.

3.3.4 (Optional Bonus) What is the difference between Pearson correlation and MSE? When would you use one over the other? How do they relate to each other?

3.4 Task 4 – From Regression to a “Toy” Classifier

One of the possible **downstream tasks** of gene-expression models is to predict disease risk based on gene expression. You have already built a **linear regression** model to predict a continuous target gene expression value (scalar), in real clinics you often predict a risk score or probability of a *binary* outcome (“disease / no disease” - 0/1) rather than a raw gene-expression value. This is called binary classification. This correlation between prediction and clinical state could be based on clinical guidelines or expert knowledge for the specific disease and it’s biomarkers. In practice LR is not used, and instead other models and loss functions (cross-entropy) are used to predict probabilities. These **are not required** for this task.

Here we take a shortcut: we keep the linear-regression hypothesis class you just built and add on a simple *threshold* to obtain class labels. (Now, $Y \in 0, 1$ instead of $Y \in \mathbb{R}$.) The goal is to see how model choice + decision threshold jointly influence downstream accuracy.

The true target y_i you used so far is assumed to be a quantitative **biomarker** measuring risk for a certain disease. We convert it to a binary label:

A. For all of the data, compute the **median** m of the target gene (across all samples).

B. Define

$$\tilde{y}_{i,\text{binary}} = \begin{cases} 1 & \text{if } y_i \geq m \\ 0 & \text{otherwise.} \end{cases}$$

Where:

- Y is the vector of target gene expression values for all samples, i.e. $Y = [y_1, y_2, \dots, y_n]$.
- y_i is the target gene expression value for sample i . ($Y[i] = y_i$)
- $\tilde{y}_{i,\text{binary}}$ is the binary label for sample i .
- m is the median of the target gene expression values (Y) across all samples.

For example, given: $Y = [1.2, 3.4, 2.5, 4.1, 2.0]$ we have $m = 2.5$, so $y_2 = 3.4$ and $y_{2,\text{binary}} = 1$, and in total the binary labels would be $Y_{\text{binary}} = [0, 1, 1, 1, 0]$.

(In real datasets you would get 0/1 labels through medical records, but here we use instead a simple threshold based on the median value of the target gene expression.)

C. Write code to compute and save Y_{binary} , a vector of 0/1 labels for each sample, where $Y_{\text{binary}}[i] = y_{i,\text{binary}}$ for the i -th sample.

D. Modify your model to predict the binary label you just created Y_{binary} which will be used as the ground-truth instead of the original continuous Y .

D.1 Modify the model for binary labels: Think what should be changed so that its predictions are 0 / 1 classes that match the labels you just defined. Follow the following steps:

D.2 Predict on the existing test split: for every sample output a prediction (0/1).

D.3 Evaluate with three classification metrics (See definitions below):

- **Accuracy** – fraction of correct predictions
- **Precision** – $TP/(TP + FP)$
- **Recall** (sensitivity) – $TP/(TP + FN)$

Definitions:

- **TP (True Positive):** Number of samples where the model predicted 1 (disease) and the true label is also 1.
- **FP (False Positive):** Number of samples where the model predicted 1 (disease) but the true label is 0 (no disease).
- **TN (True Negative):** Number of samples where the model predicted 0 (no disease) and the true label is also 0.
- **FN (False Negative):** Number of samples where the model predicted 0 (no disease) but the true label is 1 (disease).
- **Accuracy:** Fraction of correct predictions

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision:** Fraction of positive predictions that are correct

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall (Sensitivity):** Fraction of actual positives that are correctly identified

$$\text{Recall} = \frac{TP}{TP + FN}$$

3.4.1 How did you modify your model for binary prediction and why?

3.4.2 How do these metrics compare? Based on what our downstream task, which metric is the most important? why?

3.5 Task 5 – Theoretical questions

Answer concisely (1-2 lines) in your PDF report:

3.5.1 Did you encounter underfitting or overfitting? How do we detect overfitting and underfitting in machine learning?

3.5.2 What would happen if we included non-linear features (e.g. x_1^2 , $\log(x_2)$, etc.) in the model? Would it help or hurt the model performance? Why? (Theoretical question, no need to implement it.)

3.5.3 Suppose we double the number of input genes, how many parameters will the model have?

3.5.4 Suppose we double the number of output/target genes, how many parameters will the model have?

3.5.5 Suppose we double the number of input samples, how many parameters will the model have?

3.5.6 Suppose we have made all modifications in questions 3.5.3–3.5.5, write down the loss equation. In your answer state the dimensions of each term (scalar, vector (and what size) and matrix (and what size)).

3.5.7 Mention two limitations of linear regression models.

3.5.8 Does the test set allow us to estimate our in-sample error?

3.5.9 Would any split of the data into two disjoint subsets result in an equally useful train-test split?

3.5.10 How did you use LLMs in this assignment? What were the benefits and limitations of using LLMs for this assignment?

4 Submission Guidelines

Submit a single ZIP file with the following structure:

HW1_ID1_ID2.zip

```
src/                                # your code files
    e.g. main.py or main.ipynb
    any other code files for reproducibility...

llm_chats/                          # exported LLM conversations (see note below)

report_ID1_ID2.pdf    #    3 pages + charts
```

Do not include the `gene_expression_regression.csv` file in the submission, as it is very large and already available for download.

New this year — LLM chat logs. If you used LLMs (e.g. ChatGPT, Claude, Gemini, Copilot) at any stage, export the full conversation(s) and place them in the `llm_chats/` folder (PDF, Markdown, or a `.txt` with shareable links).

4.1 Written report guidelines

In essence, the goal of the report is to be a *concise* summary of your work, it should show your understanding of the model, results and reasoning behind your decisions. No need to include code, any equations or algorithms used should be mentioned if we saw them in class or explained otherwise.

- **Content:**
 - Include your names and IDs on the first page
 - Include answers and results of all parts (1 – 5) in the report.
 - Include all plots and figures that help explain your results.

- Use clear markers for each task and question.
- **Length:** Maximum **3 pages** (excluding charts and figures, equations count towards length).
- **Font:** Readable (e.g., Arial) with a minimum size of 11pt and standard margins.
- **Explanations:** Provide clear, short explanations for your model choices, esp. if they diverge from what we've seen in class. Include any challenges you faced and how you solved them.
- **Charts:** Include relevant figures, make sure they are clear and labeled, and provide a brief explanation of how it relates to your results.

5 Grading rubric

Component	Pts
Written Report	30
Full implementation of the model (Sections 3.1–3.4)	25
Written answers (3.5)	25
Model performance	15
Results reproducibility	5

6 Academic integrity & help

- Do not use any external libraries for the model implementation, except **numpy** and **pandas**.
- Do not use **sklearn** for the model implementation, only for data splitting.
- Use any online documentation or LLMs, but **do not share code or prompts** across pairs.
- Post questions in the **HW1 Moodle forum**. For Private issues email the TA.

Good luck!

7 FAQ

Q: Can I use Kaggle/Colab? A: You may use any platform for development, but your final submission should follow the required folder structure and include files as specified.

Q: What libraries can I use? A: You can use `numpy` and `pandas` for data manipulation, but do not use any libraries for the model implementation, except for data splitting with `sklearn.model_selection.train_test_split`. For plotting, any library is fine (e.g., `matplotlib`, `seaborn`, etc.).

Q: Do I need a GPU to run this assignment? A: No, you can run this assignment without it (using a CPU). The dataset is small enough to fit in memory and the model training will not take long.

Q: What language should I use for the report and code? A: For the report - Hebrew or English (English is recommended since you will use many technical terms anyway). Any language is fine for the code!

Q: What should I do if my results are different each run? A: Make sure to set all random seeds (e.g., `random_state=42` in `train_test_split` and for any random number generators you use) for reproducibility.

Q: What Loss function should I use? A: You can use any loss function that is suitable, but the most common one for regression tasks is Mean Squared Error (MSE).

Q: What does code reproducibility mean? A: Code reproducibility means that anyone who runs your code and provides the data should be able to obtain the same results as you did. This includes using the same random seeds, data splits, and ensuring that all necessary files are included in the submission.